

Data Science Program
The George Washington University

Capstone Report
Capstone Report – Fall 2025

**House Price Prediction and Forecasting
using a ZIP $\times$ Month TAB–MTL Quantile Model**

Ujjawal Dwivedi
M.S. in Data Science

supervised by
Prof. Amir Jafari

Contents

1	Introduction	4
2	Data and Preprocessing	5
2.1	Raw dataset	5
2.2	Downcasting and type normalization	5
3	Price History Flattening	6
3.1	Motivation	6
3.2	CombinedEventsBuilder	6
3.3	Base snapshot	7
4	ZIP×Month Index Construction	7
4.1	Canonical event date and filtering	7
4.2	Listing vs sold events	7
4.3	Raw ZIP index	8
4.4	Reliability weighting	8
4.5	County and state pooling	9
4.6	Macro medians	9
4.7	Future labels: returns on the ZIP index	9
5	Geo-Tiling and Feature Engineering	10
5.1	Approximate H3 geo-tiling	10
5.2	Temporal and time-series features	10
5.3	Macro lags	11
5.4	Final feature set	11
6	Forecasting Formulation and Splitting	12
6.1	What is forecasting here?	12
6.2	Train/validation/holdout split	12
6.3	Winsorization of labels	13
7	Model Architecture and Training	13
7.1	Tabular multi-task quantile network	13
7.2	Loss function: pinball + L1 median	14
7.3	Optimization and training schedule	15
7.4	From returns to price levels	15
7.5	Prediction suppression (confidence gating)	15
8	Evaluation Metrics	15
9	Results	16
9.1	Main holdout performance	16
9.2	Suppression / confidence gating	17
9.3	Rolling 60-day backtest	17
9.4	Visualizations	18

10 Discussion	19
10.1 Interpretation of horizons	19
10.2 Effect of reliability weighting	19
10.3 Strengths of the approach	19
10.4 Limitations	19
11 Future Work	19
12 Conclusion	20

1 Introduction

Forecasting house prices is a central problem for home builders, lenders, and buyers. For a builder, decisions about land acquisition, community launch timing, and pricing strategy depend on where local home values are likely to move over the next several months. For individual buyers, the same forecasts determine whether “now” is a good time to buy or wait.

In this capstone, I build an end-to-end forecasting pipeline that takes raw web-scraped listing histories and macroeconomic indicators and produces probabilistic forecasts of home price levels at a $ZIP \times month$ granularity. The core model is a **tabular neural network** trained in a **multi-task** and **quantile regression** setting (TAB-MTL quantile model). Concretely, the model predicts the distribution of future index returns for each ZIP code at two horizons:

- $h = 1$ month ahead (H1)
- $h = 2$ months ahead (H2)

Rather than directly predicting prices for individual listings, I first construct a $ZIP \times month$ *index* that represents the typical sale or listing price in that ZIP and time period. There are several reasons for this design choice:

1. **Data sparsity and noise at the listing level.** Individual listing prices are noisy and strongly affected by micro-level heterogeneity (lot premium, upgrades, incentives, staging). Aggregating to median prices within a ZIP and month creates a smoother time series that is more appropriate for forecasting.
2. **Alignment with business decisions.** Builder pricing and land strategy decisions are typically taken at the neighborhood or ZIP level rather than for one specific MLS listing. A ZIP-level index connects more directly to those decisions.
3. **Statistical reliability.** By aggregating transactions within a period we can compute reliability measures (e.g., number of sales, number of months of history) and use them as weights in the loss function, which improves stability of the model.

Once a ZIP-level index is defined, I construct a time series for each ZIP from January 2022 onward and learn a model that observes the current index level, its lagged values and momentum, geospatial features, and macroeconomic indicators. The model outputs *quantiles* of the future log-index change, which are then transformed back into level predictions and prediction intervals. This allows us not only to produce point forecasts, but to quantify uncertainty.

This report describes the system in detail:

- how raw Zillow-style listing histories are flattened into *events*;
- how a $ZIP \times month$ index is constructed, including county and state pooling and reliability weighting;
- how geospatial “geo-tiling” features approximate H3 cells;
- how time-series features (lags, momentum, volatility, macro lags) are engineered;
- the architecture and training of the tabular multi-task quantile network;
- experimental setup and evaluation metrics; and

- results on a fixed holdout and a rolling backtest.

Throughout, I focus on being explicit about design decisions and hyperparameters so that the pipeline can be reproduced or extended in future work.

2 Data and Preprocessing

2.1 Raw dataset

The starting point is a CSV export (here called `sub_sample.csv`) of a larger scraped dataset derived from online real-estate listings. Each row represents a snapshot of a property (identified by a Zillow-style ID ZPID) at a particular scrape time. The key groups of columns are:

- **Identifiers and location.** ZPID, street address, city, state, county, ZIP code, FIPS and county codes, latitude and longitude.
- **Home attributes.** Bedrooms, bathrooms, square footage, lot size, year built, home type, construction materials, presence of pool, parking, etc.
- **School and neighborhood attributes.** Distances and ratings for elementary, middle and high schools; demographic and economic statistics by ZIP (population, income, median age, growth metrics).
- **Market indicators.** Days on market, median listing price, hotness score, supply and demand scores, weekly average mortgage rate, and unemployment rate.
- **History JSON.** A PRICEHISTORY column containing a JSON array of past events for that property (listing, price change, sold, pending, etc.), with timestamps and prices.

In the offline experiment used for this report, the raw CSV contains approximately 8.6×10^5 rows and 150+ columns. After flattening the JSON history we obtain roughly 5.0×10^5 price events from January 2022 onward.

2.2 Downcasting and type normalization

For memory efficiency, all numeric columns are *downcast* to the smallest feasible numeric dtype:

- floating-point columns are converted to `float32`;
- integer columns are downcast to `int32` or `int16` when possible;
- object columns with moderate cardinality (less than 40% unique values) are cast to `category`.

Several helper functions are used to coerce messy text representations into numeric values. For example, `safe_to_double` strips commas, currency symbols and percent signs from strings like “\$1,234” or “5%” before parsing them into floats. Boolean flags are normalized using `safe_to_binary_from_text` and `safe_to_binary_from_number`.

This step ensures that subsequent grouping and aggregation operations in pandas are both memory-efficient and numerically stable.

3 Price History Flattening

3.1 Motivation

Zillow-style datasets often contain a rich `PRICEHISTORY` JSON list per property, where each element corresponds to a historical event such as:

- initial listing at a given price;
- subsequent price changes;
- contingent or pending status;
- final sale event with closing price.

To construct an index over time we must operate at the *event* level, rather than at the snapshot level. The `CombinedEventsBuilder` class implements this flattening.

3.2 CombinedEventsBuilder

Given a base `DataFrame` with one row per property snapshot and a JSON column `PRICEHISTORY`, the builder performs the following steps:

1. **Select core columns.** Keep only `ZPID` and `PRICEHISTORY` for flattening, while preserving the full base `DataFrame` for later use as a “snapshot” of property attributes.
2. **Parse JSON safely.** For each row, parse the `PRICEHISTORY` string into Python objects. If parsing fails, the row is dropped from the event table.
3. **Explode events.** Use `DataFrame.explode` to create one row per event, with a `JSON_INDEX` indicating the position of the event in the history list.
4. **Extract event fields.** For each event dictionary, extract:
 - **date:** calendar date (coerced to `datetime64`), stored as `evt_date`;
 - **time:** millisecond UNIX timestamp, used to create a high-resolution `evt_ts`;
 - **event:** event type string (lowercased) such as “listed for sale”, “price change”, “sold”, “pending sale”;
 - **price:** event price, cleaned and parsed as floating-point `evt_price`;
 - **pricePerSquareFoot:** if available, parsed as `evt_price_psf`;
 - **postingIsRental:** a Boolean flag indicating rental listings, converted to `evt_is_rental`;
 - **source, attributeSource.infoString1, attributeSource.infoString2:** extended meta-data about the data source and MLS, used as `evt_source`, `evt_mls_id` and `evt_mls_name`.
5. **Canonical event timestamp.** For each event we define `sort_ts` as the best available timestamp:

$$\text{sort_ts} = \begin{cases} \text{evt_ts}, & \text{if available} \\ \text{evt_date}, & \text{otherwise.} \end{cases}$$

Events with neither a valid date nor timestamp are discarded.

6. **Event sequencing.** Within each ZPID, events are sorted by `evt_date`, `sort_ts` and `JSON_INDEX`. A running event sequence number `event_seq` is assigned, along with:

- `days_since_prev`: difference in days from the previous event; and
- `days_since_first`: difference in days from the first event for that property.

The result is an *event-level* table with roughly 5×10^5 rows in the current experiment.

3.3 Base snapshot

The event table is then merged back with a base snapshot of property attributes. For each ZPID, we select the latest snapshot according to the scrape timestamp `SCRAPEDAT`. This snapshot provides consistent values for location, home type, size, demographic statistics, and market features that are treated as fixed covariates for all events associated with that property.

4 ZIP×Month Index Construction

4.1 Canonical event date and filtering

The `ZipMonthIndexBuilder` operates on the combined events table and produces a panel of ZIP×month indices. Key steps are:

1. **Canonical date.** A canonical event date `EVT_DATE` is defined as:

$$\text{EVT_DATE} = \begin{cases} \text{EVT_DATE}, & \text{if already present} \\ \text{evt_date}, & \text{otherwise,} \end{cases}$$

coerced to a calendar date.

2. **Date filter.** Only events from January 1, 2022 onward are retained. In the current run this leaves 503,153 events.
3. **Non-rental filter.** Events with `EVT_IS.RENTAL` flagged are dropped, leaving 487,223 non-rental events.

Each event is assigned to a year-month bucket:

$$\text{YM} = \text{EVT_DATE truncated to month.}$$

4.2 Listing vs sold events

Event types are normalized to lower case and mapped into broad categories:

- **Listing-like:**

`{listing, for sale, listed for sale, price change}`

capture asking prices.

- **Sold-like:**

`{sold, sale, closed}`

capture realized transaction prices.

The price field is aggressively cleaned using the same regular expression logic as `safe_to_double` (strip currency symbols, commas, whitespace) and then parsed to a float `EVT_PRICE`.

For each key

(ZIPCODE, STATE_MODE, COUNTY_MODE, YM)

we compute:

$$\begin{aligned} N_SOLD &= \#\{\text{sold-like events with a price}\}, \\ SOLD_MEDIAN &= \text{median}(EVT_PRICE \mid \text{sold-like}), \\ N_LIST &= \#\{\text{listing-like events with a price}\}, \\ LIST_MEDIAN &= \text{median}(EVT_PRICE \mid \text{listing-like}). \end{aligned}$$

In the experiment, the aggregation yields 53,516 ZIP $\times$ month rows.

4.3 Raw ZIP index

The *raw* ZIP index for each key is defined as:

$$IDX_RAW_{z,t} = \begin{cases} SOLD_MEDIAN_{z,t}, & \text{if at least one sale is observed,} \\ LIST_MEDIAN_{z,t}, & \text{otherwise.} \end{cases}$$

This produces a typical sale/listing price per month. Optionally, thresholds on N_SOLD and N_LIST can be enforced, but in this experiment they are left relaxed in order to retain more ZIPs and backfilling is handled via county/state pooling.

Each ZIP’s time series is sorted by month and a running count `N_MONTHS_TO_DATE` is computed, representing the number of months (i.e., index points) observed so far for that ZIP.

4.4 Reliability weighting

Not all ZIP-month observations are equally reliable. A ZIP with 20 months of history and 50 transactions in the current month should be trusted more than a ZIP with a single listing and no sales.

To capture this, I define a reliability weight:

$$w_{z,t} = \min\{w_{\text{hist}}(z, t) \cdot w_{\text{tx}}(z, t), 1.0\},$$

where:

$$\begin{aligned} n_{\text{tx}}(z, t) &= \max(N_SOLD, N_LIST), \\ w_{\text{hist}}(z, t) &= \frac{\log(1 + \min(N_MONTHS_TO_DATE, 24))}{\log(1 + 12)}, \\ w_{\text{tx}}(z, t) &= \max\left(\frac{\log(1 + n_{\text{tx}}(z, t))}{3}, 0.2\right). \end{aligned}$$

Intuitively, w_{hist} rewards ZIPs with longer history (up to two years), and w_{tx} rewards months with more transactions. Both are bounded below by 0.2 to avoid completely discarding new or thin ZIPs. The combined weight $w_{z,t}$ is capped at 1.

This weight is used later in the loss function to emphasize reliable H1 labels (Section 7).

4.5 County and state pooling

Some ZIP×month combinations may have no transactions and no active listings, resulting in missing `IDX_RAW`. Instead of dropping these months, we borrow strength from higher-level geographies:

- For each county–month pair (c, t) we compute `IDX_COUNTY_MEDc,t`, the median `IDX_RAW` across ZIPs in that county.
- For each state–month pair (s, t) we compute `IDX_STATE_MEDs,t`, the median `IDX_RAW` across ZIPs in that state.

An *effective* index is then defined as:

$$\text{IDX_EFF}_{z,t} = \text{coalesce}\left(\text{IDX_RAW}_{z,t}, \text{IDX_COUNTY_MED}_{c(z),t}, \text{IDX_STATE_MED}_{s(z),t}\right),$$

and we simply write:

$$\text{IDX}_{z,t} = \text{IDX_EFF}_{z,t}.$$

In addition, relative indices are computed:

$$\begin{aligned} \text{IDX_REL_COUNTY}_{z,t} &= \frac{\text{IDX}_{z,t}}{\text{IDX_COUNTY_MED}_{c(z),t}}, \\ \text{IDX_REL_STATE}_{z,t} &= \frac{\text{IDX}_{z,t}}{\text{IDX_STATE_MED}_{s(z),t}}. \end{aligned}$$

These capture whether a ZIP is relatively expensive or cheap compared to its county or state peer group.

4.6 Macro medians

For each ZIP×month key, the builder also aggregates macroeconomic variables (if present in the event table) by median:

- `WEEKLY_AVERAGE_MORTGAGE_RATE`
- `UNEMPLOYMENT_RATE`

These medians are merged into the ZIP index table and later used to construct lagged macro features.

4.7 Future labels: returns on the ZIP index

Forecasting is formulated in terms of *returns* rather than levels. For each ZIP z and month t we define:

$$\begin{aligned} \text{IDX_FUTURE_H1}_{z,t} &= \text{IDX}_{z,t+1}, \\ \text{IDX_FUTURE_H2}_{z,t} &= \text{IDX}_{z,t+2}. \end{aligned}$$

The targets are the $\Delta \log$ returns:

$$\begin{aligned} Y_H1(z, t) &= \log(1 + \text{IDX_FUTURE_H1}_{z,t}) - \log(1 + \text{IDX}_{z,t}), \\ Y_H2(z, t) &= \log(1 + \text{IDX_FUTURE_H2}_{z,t}) - \log(1 + \text{IDX}_{z,t}). \end{aligned}$$

Working in log space has two advantages:

- it stabilizes variance and makes the distribution of returns more symmetric; and
- for small changes, $\Delta \log$ is approximately the percentage change in price.

These two labels are the primary targets for the neural network (one head per horizon).

5 Geo-Tiling and Feature Engineering

5.1 Approximate H3 geo-tiling

Real H3 (hexagonal hierarchical geospatial indexing) requires an external C library, which is not available in the constrained compute environment of this project. Instead, I implement an approximate H3-style tiling using lat/long rounding.

The `build_geo_tiling` function:

1. Aggregates the combined events table to ZIP×month by computing median latitude and longitude and the mode of state and county.
2. For a set of resolutions $r \in \{6, 7, 8, 9\}$, defines a pseudo-H3 ID:

$$\text{H3_R}r = \text{"H3R"} + r + \text{LATITUDE}_{\text{median}}^{(3\text{dp})} + \text{"-"} + \text{LONGITUDE}_{\text{median}}^{(3\text{dp})}.$$

The resulting columns `H3_R6`, `H3_R7`, `H3_R8`, `H3_R9` are treated as categorical features that approximate the spatial cell of each ZIP at multiple resolutions. They allow the model to capture spatial autocorrelation beyond just ZIP, county and state.

5.2 Temporal and time-series features

A dedicated `ZipIndexFeatureizer` class builds time-series features on top of the ZIP index table. The main components are:

Global month index and seasonality. Each month is converted into an integer index relative to an origin (January 2015):

$$\text{GLOBAL_MONTH_INDEX} = (\text{year} - 2015) \times 12 + (\text{month} - 1).$$

To capture annual and semi-annual seasonal patterns, I add sinusoidal features:

$$\begin{aligned} \text{SEAS_SIN_12} &= \sin\left(2\pi \frac{\text{GLOBAL_MONTH_INDEX}}{12}\right), \\ \text{SEAS_COS_12} &= \cos\left(2\pi \frac{\text{GLOBAL_MONTH_INDEX}}{12}\right), \\ \text{SEAS_SIN_6} &= \sin\left(2\pi \frac{\text{GLOBAL_MONTH_INDEX}}{6}\right), \\ \text{SEAS_COS_6} &= \cos\left(2\pi \frac{\text{GLOBAL_MONTH_INDEX}}{6}\right). \end{aligned}$$

Lagged index levels and momentum. For each ZIP, lagged values of the index are computed:

$$\begin{aligned}\text{IDX_LAG_1} &= \text{IDX}_{t-1}, \\ \text{IDX_LAG_3} &= \text{IDX}_{t-3}, \\ \text{IDX_LAG_6} &= \text{IDX}_{t-6}.\end{aligned}$$

Momentum and short-term returns are defined as:

$$\begin{aligned}\text{IDX_PCT_D1} &= \frac{\text{IDX}_t}{\text{IDX_LAG_1}} - 1, \\ \text{IDX_MOM_3} &= \frac{\text{IDX}_t}{\text{IDX_LAG_3}} - 1, \\ \text{IDX_MOM_6} &= \frac{\text{IDX}_t}{\text{IDX_LAG_6}} - 1.\end{aligned}$$

These capture the recent trend in the ZIP index.

Rolling volatility. To quantify local volatility, a rolling standard deviation over the last three and six months is computed for each ZIP:

$$\begin{aligned}\text{IDX_ROLL_STD_3} &= \text{std}(\text{IDX}_{t-2:t}), \\ \text{IDX_ROLL_STD_6} &= \text{std}(\text{IDX}_{t-5:t}).\end{aligned}$$

5.3 Macro lags

If the ZIP index table contains macro variables such as `WEEKLY_AVERAGE_MORTGAGE_RATE` and `UNEMPLOYMENT_RATE`, they are renamed to `MORTGAGE_RATE_M` and `UNEMPLOYMENT_RATE_M` and lagged:

For each base macro variable X_t we compute:

$$\begin{aligned}X_L1 &= X_{t-1}, & X_D1 &= X_t - X_{t-1}, \\ X_L3 &= X_{t-3}, & X_D3 &= X_t - X_{t-3}, \\ X_L12 &= X_{t-12}, & X_D12 &= X_t - X_{t-12}.\end{aligned}$$

These capture both levels and slopes of macroeconomic conditions, allowing the model to learn that, for example, rising mortgage rates may dampen future price growth.

5.4 Final feature set

After feature engineering, and after excluding non-feature key columns and direct label columns, the dataset used for modeling consists of:

- **Numeric features (≈ 28).** Current index level, lagged levels and momentum, rolling volatility, seasonal sin/cos terms, macro levels and lags, relative county and state indices, and reliability weight.
- **Categorical features (3 in this run).** A subset of $\{\text{ZIPCODE}, \text{STATE_MODE}, \text{COUNTY_MODE}, \text{H3_R6}, \text{H3_R7}, \text{H3_R8}, \text{H3_R9}\}$, encoded via embeddings. Only columns present in the data are used; in the reported experiment we end up with 3 categorical columns.

All numeric features are standardized using a `StandardScaler` fit on the training split. Categorical features are mapped to integer IDs based on the unique values observed in the training data only; unseen categories in validation or holdout are mapped to a generic “unknown” index.

6 Forecasting Formulation and Splitting

6.1 What is forecasting here?

In this project, *forecasting* means predicting future values of a time-indexed quantity—the ZIP index—based on its own history and other explanatory variables. Concretely:

- For each ZIP code z and month t with index level $\text{IDX}_{z,t}$, we wish to estimate the distribution of $Y_H1(z, t)$ and $Y_H2(z, t)$, the 1- and 2-month ahead index returns.
- The model uses all information available up to month t : lagged indices, volatility measures, macro lags, geospatial context, and seasonality indicators.

Compared to pure point forecasting (predicting only the expected future index), quantile forecasting outputs multiple quantiles (e.g., 10th, 50th, 90th percentiles) of the future outcome, which can be converted into prediction intervals. This allows the end user to see not just “our best guess” but also how uncertain that guess is.

6.2 Train/validation/holdout split

Although the code originally contains a day-based holdout split, the final experiment uses a *month-based* split that is more interpretable in a time-series context.

Let YM denote the year-month of a row. Among all rows with valid labels (Y_H1 , Y_H2 , and IDX non-missing), we identify the latest month YM_{last} , then define:

$$\begin{aligned} m_0 &= \text{YM}_{\text{last}}, \\ m_1 &= \text{YM}_{\text{last}} - 1 \text{ month}, \\ m_2 &= \text{YM}_{\text{last}} - 2 \text{ months}, \\ m_3 &= \text{YM}_{\text{last}} - 3 \text{ months}. \end{aligned}$$

The final splits are:

- **Holdout.** Months m_0 and m_1 (most recent 1–2 months).
- **Validation.** Months m_2 and m_3 .
- **Training.** All months strictly earlier than m_3 .

In the reported run:

- training: 45,622 labeled rows;
- validation: 2,372 labeled rows;
- holdout: 1,373 labeled rows.

If, in some dataset, the training sample constituted less than 25% of all labeled rows, a fallback schedule using only one month for holdout and one month for validation would be used; in this experiment that fallback was not needed.

6.3 Winsorization of labels

To reduce the impact of extreme outliers in the target distributions, I apply *winsorization* separately by `STATE_MODE` and month, but only based on the training portion of the data.

For each state-month group:

1. compute the 25th and 75th percentiles (Q_1 , Q_3) of the training labels (e.g., `Y_H1`);
2. compute the interquartile range $\text{IQR} = Q_3 - Q_1$;
3. define lower and upper fences:

$$\text{lo} = Q_1 - k \cdot \text{IQR}, \quad \text{hi} = Q_3 + k \cdot \text{IQR},$$

with $k = 1.5$ for H1 and $k = 3.0$ for H2;

4. clip all labels (training, validation, holdout) to this range and store in new columns `Y_H1_WZ` and `Y_H2_WZ`.

The model then uses whichever of `Y_H1` or `Y_H1_WZ` is available. This approach preserves the ordinal structure of extreme events while preventing a handful of outliers from dominating the loss.

7 Model Architecture and Training

7.1 Tabular multi-task quantile network

The forecasting model is a **tabular neural network** implemented in PyTorch. “Tabular” here means that the input is a mixture of numeric and categorical features rather than images or text. The architecture is multi-task (two horizons) and performs quantile regression.

Embeddings for categorical features

Each categorical column c has a finite vocabulary of size K_c obtained from the training set. Instead of one-hot encoding, we use an embedding table:

$$E_c \in \mathbb{R}^{(K_c+1) \times d_c},$$

where d_c is the embedding dimension and the additional row corresponds to padding / unknown. The dimension is chosen as:

$$d_c = \min(\text{emb_cap}, \max(4, 4 \cdot \lfloor K_c^{1/4} \rfloor)),$$

with `emb_cap` = 64. The embeddings for each categorical feature are concatenated to form a dense representation of the categorical part of the input.

Numeric features

All numeric features are standardized to zero mean and unit variance using statistics from the training set. They are then fed directly into the network.

Trunk MLP

The numeric and embedded categorical vectors are concatenated to form the input to a multi-layer perceptron (MLP) trunk. If D is the total input dimension, the trunk is:

$$h_0 = x \in \mathbb{R}^D, \quad h_{l+1} = \text{Dropout}(\text{ReLU}(W_l h_l + b_l)), \quad l = 0, \dots, L-1,$$

with:

- HIDDEN = 384 neurons per layer;
- LAYERS = 3 hidden layers;
- DROPOUT = 0.15 for regularization.

Heads and outputs

The model has two output heads, one per horizon (H1 and H2). Each head is a linear layer mapping from the trunk dimension to Q outputs, where $Q = 3$ is the number of quantiles. The quantiles used are:

$$\tau \in \{0.1, 0.5, 0.9\}.$$

Thus, for each sample and each horizon the network outputs three numbers: estimated 10th, 50th (median), and 90th percentiles of the log-return distribution.

7.2 Loss function: pinball + L1 median

For a given target y and predicted quantiles $\hat{y}_\tau$, the *pinball loss* for quantile τ is:

$$\rho_\tau(u) = \max\{\tau u, (\tau - 1)u\}, \quad u = y - \hat{y}_\tau.$$

The total quantile loss for a head is:

$$L_{\text{pinball}} = \frac{1}{Q} \sum_{\tau \in \{0.1, 0.5, 0.9\}} \mathbb{E}[\rho_\tau(y - \hat{y}_\tau)].$$

To stabilize the median prediction, we add an L1 penalty on the median (0.5 quantile) output:

$$L_{\text{median}} = \mathbb{E}[|y - \hat{y}_{0.5}|].$$

The combined loss for horizon h is:

$$L^{(h)} = \lambda_{\text{pin}} L_{\text{pinball}}^{(h)} + \lambda_{\text{med}} L_{\text{median}}^{(h)},$$

with PINBALL_WEIGHT = 1.0 and L1_MEDIAN_WEIGHT = 0.5.

For the H1 head, this loss is further reweighted by the reliability weight $w_{z,t}$ from Section 4:

$$\tilde{L}^{(\text{H1})} = \mathbb{E}[w_{z,t} \cdot L_{z,t}^{(\text{H1})}].$$

This means the model pays more attention to reliable ZIP×month labels when learning H1 forecasts.

7.3 Optimization and training schedule

The model is trained using:

- **Optimizer:** AdamW with learning rate 2×10^{-3} and weight decay 10^{-4} .
- **Batch size:** BATCH_SIZE = 2048.
- **Max epochs:** EPOCHS = 60.
- **Early stopping:** monitor the sum of validation MAE for H1 + H2; stop if no improvement for PATIENCE = 8 epochs.

In the reported main experiment, early stopping triggers at epoch 23, with the best model observed at epoch 15. Training runs on CPU in this offline setting (**Device:** cpu), but the architecture is compatible with GPU training.

7.4 From returns to price levels

During evaluation, predicted quantiles in log–return space are transformed back into price levels using:

$$\hat{P}_\tau = \exp(\log(1 + \text{IDX}_{z,t}) + \hat{y}_\tau) - 1,$$

where $\hat{y}_\tau$ is the predicted quantile of $\Delta \log$ index.

To make loss and metrics interpretable in dollar terms, a simple scaling heuristic is used: if the median index value in a split is less than 10,000, a scale factor of 1,000 is applied, i.e., predicted levels are multiplied by 1,000 before computing MAE and related metrics.

7.5 Prediction suppression (confidence gating)

In many real applications, it is better to return “no forecast” for cases where the model is highly uncertain. The prediction intervals naturally provide a measure of uncertainty.

For each prediction we compute the relative width of the 10–90 interval:

$$\text{rel_width} = \frac{\max(|P_{0.9} - P_{0.1}|, \epsilon)}{\max(|P_{0.5}|, \text{MIN_LEVEL})},$$

with MIN_LEVEL = 1.0. A prediction is *suppressed* if

$$\text{rel_width} > \text{SUPPRESS_WIDTH_PCT},$$

with SUPPRESS_WIDTH_PCT = 0.15 in this experiment. Suppression statistics are reported in Section 9.2.

8 Evaluation Metrics

For each horizon and data split (validation, holdout, rolling folds) the following metrics are computed on the price–level scale:

MAE Mean absolute error in dollars:

$$\text{MAE} = \frac{1}{n} \sum_i |y_i - \hat{y}_i|.$$

R^2 Coefficient of determination:

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

WAPE Weighted absolute percentage error:

$$\text{WAPE} = \frac{\sum_i |y_i - \hat{y}_i|}{\sum_i |y_i|}.$$

MdAPE Median absolute percentage error:

$$\text{MdAPE} = \text{median}_i \left(\frac{|y_i - \hat{y}_i|}{|y_i|} \right).$$

Pct $|err| \leq 10\%$ Fraction of predictions where the absolute percentage error is at most 10%:

$$\mathbb{P}(|y_i - \hat{y}_i| \leq 0.1|y_i|).$$

PI cover Coverage of the 10–90 prediction interval:

$$\mathbb{P}(y_i \in [P_{0.1,i}, P_{0.9,i}]).$$

Rel. width Mean relative width of the prediction interval:

$$\mathbb{E} \left[\frac{|P_{0.9,i} - P_{0.1,i}|}{\max(|P_{0.5,i}|, \text{MIN_LEVEL})} \right].$$

Together, these metrics describe both *accuracy* (MAE, WAPE, MdAPE, Pct $|err| \leq 10\%$, R^2) and *calibration/uncertainty* (PI cover, relative width).

9 Results

9.1 Main holdout performance

The main experiment trains on data up to three months before the last labeled month, uses the next two months as validation, and evaluates on the final 1–2 months as a fixed holdout. Table 1 summarizes the results.

Table 1: Main holdout performance (last 1–2 months) for the ZIP×Month TAB–MTL quantile model. All metrics are computed on the dollar scale after transforming from log–returns.

Metric	H1 (1-month ahead)	H2 (2-month ahead)
MAE	31.73	31.27
R^2	0.9971	0.9960
WAPE	0.0424	0.0428
MdAPE	0.0311	0.0294
Pct $ err \leq 10\%$	0.8093	0.8232
PI cover	0.9803	0.9774
Rel. width	0.8913	0.8500

Both horizons achieve very strong performance, with MAE around \$32, WAPE around 4.3%, and more than 80% of predictions landing within $\pm 10\%$ of true prices. The R^2 values above 0.995 indicate that the model captures the vast majority of cross-sectional and temporal variation in the ZIP-level index.

The prediction intervals are slightly wider than ideal (relative width around 0.85–0.89), but in exchange they provide excellent coverage close to 98% of true outcomes.

9.2 Suppression / confidence gating

Table 2 shows how often predictions are suppressed on the holdout set when using the relative-width threshold of 0.15.

Table 2: Suppression / confidence gating statistics on the main holdout for the ZIP×Month TAB-MTL quantile model.

Horizon	Suppressed predictions	Total evaluated	Suppression rate
H1 (1-month ahead)	1038	1369	0.7582
H2 (2-month ahead)	929	1369	0.6786

The suppression rates are relatively high (68–76%), reflecting a conservative threshold on interval width. In practice, this means that the system only returns forecasts when the 10–90 interval is relatively tight compared to the predicted price level, i.e., when the model is confident. The remainder of cases can be flagged for fallback heuristics or manual review.

9.3 Rolling 60-day backtest

To verify that performance is stable over time and not an artifact of a particular split, I conduct a rolling backtest with three 60-day holdout windows. For each fold:

1. choose a 60-day holdout window at the end of the available labeled period;
2. train the model on all data strictly before that window;
3. evaluate on the window.

Table 3 summarizes the metrics across the three folds.

Table 3: Rolling 60-day backtest results (3 folds, holdout windows only) for the ZIP×Month TAB-MTL quantile model.

Fold	Horizon	MAE	R^2	WAPE	MdAPE	Pct $ err \leq 10\%$	PI cover	Rel. width
1	H1	84.83	0.9738	0.0993	0.0466	0.6584	0.9096	0.9812
1	H2	71.82	0.9837	0.0872	0.0459	0.6744	0.9671	0.9317
2	H1	39.99	0.9977	0.0474	0.0411	0.7529	0.9780	0.9292
2	H2	28.76	0.9976	0.0398	0.0341	0.8286	0.9725	0.8809
3	H1	41.54	0.9932	0.0555	0.0279	0.7889	0.9737	0.7786
3	H2	28.03	0.9978	0.0383	0.0335	0.8028	0.9204	0.6639

The rolling backtest confirms that:

- Performance is consistently strong across different historical periods, with WAPE generally between 4–10%.
- H2 (two-month ahead) often achieves similar or better WAPE and MAE than H1, suggesting that much of the medium-term trend is predictable given the engineered features.
- Prediction interval coverage remains high (around 92–98%) with moderate relative widths.

9.4 Visualizations

The script also generates several figures saved under the `figs/` directory:

- `fig_mae_bar.png`: bar chart of MAE across H1 and H2, separated by holdout vs rolling folds.
- `fig_wape_bar.png`: bar chart of WAPE across horizons and splits.
- `fig_r2_rolling.png`: line plot of R^2 over folds.
- `fig_wape_robust_H1.png`, `fig_wape_robust_H2.png`: WAPE and MdAPE comparisons.
- `fig_pi_cover.png`: coverage of prediction intervals by split.
- `fig_pi_width.png`: relative prediction interval width.
- `fig_holdout_radar.png`: radar chart comparing H1 vs H2 on multiple metrics.

Example LaTeX code for including the radar chart:



Figure 1: Relative performance of H1 vs H2 on the main holdout across multiple metrics.

The radar chart in particular illustrates that H2 generally has slightly lower MAE and WAPE, while H1 tends to have marginally higher interval coverage and slightly wider prediction intervals.

10 Discussion

10.1 Interpretation of horizons

An interesting empirical observation is that the 2-month ahead horizon (H2) often performs at least as well as the 1-month horizon (H1) in terms of WAPE and MdAPE. This suggests that the *medium-term* component of ZIP-level price dynamics is smoother and more forecastable than the very short-term month-to-month noise.

H1 forecasts capture more of the high-frequency wiggles, and therefore experience slightly higher relative error and wider intervals. H2 forecasts, which effectively average over two months of future dynamics, tend to be more stable.

10.2 Effect of reliability weighting

Weighting the H1 loss by the reliability factor $w_{z,t}$ ensures that the model is primarily trained on ZIP $\times$ month observations with reasonable history and transaction volume. This reduces overfit to extremely noisy ZIPs where a single listing or sale could drastically change the median.

As a trade-off, predictions for low-reliability ZIPs become more conservative and are more likely to be suppressed by the confidence gate, which is acceptable in many production scenarios.

10.3 Strengths of the approach

- The model learns from *both* cross-sectional and time-series information, leveraging thousands of ZIPs simultaneously.
- The pipeline is fully end-to-end: starting from raw JSON price histories and ending with probabilistic forecasts.
- The quantile framework provides predictive intervals, which are particularly valuable in planning and risk management contexts.

10.4 Limitations

- The approximate H3 geo-tiling is a rough proxy and does not capture true hexagonal adjacency; using a proper H3 library could improve spatial smoothness.
- The current model works at the ZIP level only; property-level heterogeneity (e.g., lot premiums, bed/bath mix) is not explicitly modeled.
- Macro variables are limited to mortgage rates and unemployment; richer macroeconomic and local market indicators (e.g., inventory, permit data) could further improve forecasts.

11 Future Work

Several extensions are natural:

- **Hierarchical modeling.** Extend the index to a multi-level hierarchy (ZIP $\rightarrow$ county $\rightarrow$ metro area) and train a joint model that explicitly respects these relationships.
- **Property-level forecasting.** Combine the ZIP index model with a hedonic property-level model that predicts deviations from the ZIP median based on detailed property attributes.

- **Richer spatial features.** Integrate a true H3-based tiling and incorporate neighboring cell indices to capture spatial spillovers.
- **Alternative architectures.** Experiment with gradient boosting machines (e.g., LightGBM) and temporal models (e.g., Seq2Seq, Temporal Fusion Transformers) as baselines or ensemble partners.
- **Productionization.** Wrap the pipeline in a reproducible data-engineering and model-serving stack (e.g., Snowflake Snowpark + batch scoring) for regular forecast updates.

12 Conclusion

This capstone project demonstrates an end-to-end system for forecasting house prices at a ZIP×month level. Starting from noisy, event-level PRICEHISTORY JSON data, I:

1. flattened and cleaned price events;
2. constructed a ZIP×month index with county and state pooling and reliability weighting;
3. engineered temporal, macro and geospatial features, including approximate H3 tiles;
4. formulated forecasting as multi-task quantile regression on log-returns at 1- and 2-month horizons; and
5. trained a tabular neural network that achieved WAPE around 4–5% and R^2 above 0.995 on the holdout, with strong performance in rolling backtests.

The resulting model provides not only accurate point forecasts but also well-calibrated prediction intervals and a principled mechanism for suppressing low-confidence predictions. This forms a strong foundation for future house-price forecasting work and can be extended to support practical pricing and land-strategy decisions for home builders.